ALADIN documentation

System requirements:

- Linux Ubuntu >= 20.04 or MacOSX 13.6.x or Microsoft Windows >=10
- Python 3.10
- > 20Gb free diskspace
- Modern GPU with >12Gb VRAM is recommended for training
- Git, Docker, and Docker-compose for training

Tested on Linux Ubuntu 20.04, MacOSX 13.6.x, and Microsoft Windows 10 and 11 Home. CPU-only support is available, but a modern GPU will speed up training and inference substantially. Tested on NVidia GeForce 3090. ALADIN works on newer versions of Python; but, the algorithm of Jimenez-Perez et al. crashes if Python >3.10.

Windows users are advised to use Powershell. Run in administrator mode and see this StackOverflow question when you experience difficulty activating the virtual python environment (step 2). Also, when using Windows, please change the shell script extensions from `.sh` to `.bat`.

Getting started

Out-of-the-box ALADIN installation evaluated on a few demo recordings. We advise to use a clean virtual environment to prevent issues with library versions.

Preparations (~10 min)

1. Create local python environment

```
python -m venv ALADIN
```

2. Activate python environment

```
source ALADIN/bin/activate #Linux and MacOSX
.\ALADIN\Scripts\activate #Windows
```

3. Install ALADIN

```
pip install ./src
```

4. Install nnUNet

```
pip install ./nnUNet
```

5. Create models folder

```
mkdir models
```

6. Download model weights from FigShare (8.1Gb) and unzip in `/models` folder such that you have `/models/Dataset301_all_0/ClassificationTrainer__nnUNetWithClassificationPlans__1d_decoding/`
7. Set environment variable

```
export aladin_models=[absolute path to root]/models #Linux and MacOSX
$Env:aladin_models="[absolute path to root]\models" #Windows
```

Run demo (~1 min)

8. Run

```
python demo.py --case=[recording]
```

Where `[recording]` can be one of `(STANFORD1, STANFORD2, A01986, A08391)`.

Expected output

- 'aladin_result.png', a figure showing the delineation and diagnosis with on- and offsets
- 'aladin_explanation.txt', a textfile containing a detailed explanation behind the diagnosis

Note: It is possible to analyse your own files. ALADIN expects Lead II electrocardiograms formatted as WFDB files with a header file (.hea) and a data file (.dat). It only selects the first signal from the data file.

Reproduce benchmark using model checkpoints

NOTE: Use Python 3.10.x as Jimenez-Perez et al. does not work for >3.10.x

Prepare data

1. Download VALIDATION.zip from FigShare (12Mb) and extract to `/data` such that you have `/data/VALIDATION/ ...`
2. Download STANFORD.zip from FigShare (6Mb) and extract to `/data` such that you have `/data/STANFORD/ ...`
3. Download the CinC dataset using `wget`

```
cd data/CINC
./download.sh #change extension to .bat for Windows
cd ../..
```

or by downloading the zip from PhysioNet (1.4Gb) and extract it to `/data/CINC` such that you have `/data/CINC/training/ ...` .

Prepare competitors

3. Clone Jimenez-Perez github repository

```
git clone https://github.com/guillermo-jimenez/DelineatorSwitchAndCompose.git
```

4. Setup Jimenez-Perez dependencies

```
cd DelineatorSwitchAndCompose
git clone https://github.com/guillermo-jimenez/sak.git
cd ..
```

5. Install Jimenez-Perez dependencies

```
cd DelineatorSwitchAndCompose/sak
pip install . --no-deps
cd ../..
```

6. Download trained model from FigShare (251Mb) and unzip in `/DelineatorSwitchAndCompose` such that you have `/DelineatorSwitchAndCompose/TrainedModels/ ...`
7. Install additional dependencies

```
pip install -r requirements.txt
```

8. Download pretrained weights of the diagnostic models from FigShare (580Mb) and unzip in `/benchmark/weights`

Prepare benchmark

9. Make results folder

```
mkdir results
```

10. Set environment variables

```
export benchmark_data=[absolute path to /data folder]
export benchmark_results=[absolute path to /results folder] #Linux and MacOSX

$Env:benchmark_data = "[absolute path to /data folder]"
$Env:benchmark_results = "[absolute path to /results folder]" #Windows
```

Run benchmark

11. Run delineation benchmark (~ 20 min)

```
./benchmark_delineation.sh #change extension to .bat for Windows
```

12. Create latex table of delineation performance on validation set

```
python paper/generate_results_tables.py
```

13. Run diagnostic benchmark on Stanford (~ 1.5 hour)

```
./benchmark_diagnosis_STANFORD.sh #change extension to .bat for Windows
```

14. Create boxplot figures of performance on Stanford

```
python paper/boxplot-stanford.py
```

15. Run diagnostic benchmark on CinC competition datset (~ 30 hours)

```
./benchmark_diagnosis_CINC.sh #change extension to .bat for Windows
```

16. Create boxplot figures of performance on CinC competition set

```
python paper/boxplot-cinc.py
```

Expected results

- A delineation performance table in raw latex code corresponding to Table 1 of the manuscript.
- Two boxplot figures showing the benchmark performance of ALADIN, ECGFounder, ResNet, and all human cardiologists on the Stanford test set and the CinC competition set. The figures are located inside `/paper/images`

Retrain ALADIN from scratch

The following steps require a GPU with CUDA enabled.

Initialize MongoDB

NOTE: You may need to run docker with `sudo`.

1. Install Docker and Docker-compose
2. Run the MongoDB container

```
cd data/MongoDB
docker-compose up -d
```

3. Download the zipped MongoDB dump from FigShare (5.3Gb) and extract to `/data/MongoDB` such that you have `/data/MongoDB/mongodb-dump/ ...`
4. Copy the dump into the running container

```
docker cp ./mongodb-dump mongodbtest:/dump
```

5. Restore the MongoDB from a dump

```
docker exec mongodbtest mongorestore --username=root \
--password=ALADIN2025 \
--authenticationDatabase=admin /dump
```

6. Go back to root folder

```
cd ../..
```

Prepare training data

7. Create the folders required by nnUNet

```
mkdir data/nnUNet_raw
mkdir data/nnUNet_preprocessed
mkdir data/nnUNet_results
```

8. Add nnUNet environment variables

```
export nnUNet_raw=[absolute path to /data]/nnUNet_raw
export nnUNet_preprocessed=[absolute path to /data]/nnUNet_preprocessed
export nnUNet_results=[absolute path to /data]/nnUNet_results #Linux and MacOSX
$Env:nnUNet_raw = "[absolute path to /data]/nnUNet_raw"
$Env:nnUNet_preprocessed = "[absolute path to /data]/nnUNet_preprocessed"
$Env:nnUNet_results = "[absolute path to /data]/nnUNet_results"
```

9. Generate training data in nnUNet format

```
python utils/generate_nnUNet_training_set.py --datatype nnunet_train
```

10. Preprocess raw data

```
nnUNetv2_plan_and_preprocess -d 117 \
-pl UNetWithClassificationPlanner \
```

```
-preprocessor_name WithClassificationPreprocessor \
--verify_dataset_integrity
```

11. Copy `/data/nnUNet_plans/Dataset117_nnunet_17/nnUNetWithClassificationPlans.json` to `/data/nnUNet_preprocessed/Dataset117_nnunet_17` and overwrite existing file.
12. Generate training data for classification branches

```
python utils/generate_nnUNet_training_set.py --datatype classification_branches
```

13. Preprocess raw data

```
nnUNetv2_plan_and_preprocess -d 201 \
-pl UNetWithClassificationPlanner \
-preprocessor_name WithClassificationPreprocessor \
--verify_dataset_integrity
```

14. Copy `/data/nnUNet_plans/Dataset201_all_101/nnUNetWithClassificationPlans.json` to `/data/nnUNet_preprocessed/Dataset201_all_101` and overwrite existing file.

Retrain ALADIN

15. Train ALADIN using 5-fold cross-validation (~ 6 hours)

```
./train_aladin.sh #change extension to .bat for Windows
```

Run benchmarks with new models

16. Update ALADIN's environment variable to use the newly trained models.

```
export aladin_models=[absolute path to nnUNet_results] #Linux and MacOSX
$Env:aladin_models = "[absolute path to nnUNet_results]" #Windows
```

17. Run delineation benchmark with new models (~ 1 min)

```
python benchmark_delineation.py --method ALADIN \
--perarrhythmia \
--modelpaths Dataset201_all_101/ClassificationTrainer__nnUNetWithClassificationPlans__1d_decoding
```

18. Run diagnostic benchmark on Stanford with new models (~ 1 min)

```
python benchmark_diagnosis.py --method ALADIN \
--dataset STANFORD --overwrite \
--modelpaths Dataset201_all_101/ClassificationTrainer__nnUNetWithClassificationPlans__1d_decoding
```

19. Run diagnostic benchmark on CinC competition set with new models (~ 12 min)

```
python benchmark_diagnosis.py --method ALADIN \
--dataset CINC --overwrite \
--modelpaths Dataset201_all_101/ClassificationTrainer__nnUNetWithClassificationPlans__1d_decoding
```

20. Create figures (see `Reproduce benchmark using model checkpoints`)

Optional: Retrain competitor models

NOTE: The MongoDB docker instance should be running in the background. If not, run `cd data/MongoDB`, `docker-compose up -d`, and `cd ../..`.

Retrain ResNet implementation from Hannun et al.

1. Train ResNet implementation (~ 1 hour)

```
cd benchmark
python train.py --method RESNET
cd ..
```

2. Evaluate trained Resnet on Stanford(~ 10s)

```
python benchmark_diagnosis.py --method Hannun \
--dataset STANFORD --overwrite \
--modelpaths benchmark/new_weights/HannunNet_checkpoint_best.pth
```

Finetune ECGFounder by Li et al.

3. Download the `1_lead_ECGFounder.pth` checkpoint from HugginFace (370Mb) and move it to `/benchmark/weights`
4. Finetune ECGFounder (~ 30 min)

```
cd benchmark
python train.py --method ECGFOUNDER
cd ..
```

5. Evaluate finetuned ECGFounder on Stanford (~ 10s)

```
python benchmark_diagnosis.py --method ECGFounder \
--dataset STANFORD --overwrite \
--modelpaths benchmark/new_weights/ECGFounderNet_checkpoint_best.pth
```

6. Create figures (see `Reproduce benchmark using model checkpoints`)

Optional: Reproduce feature space analysis

1. Create embeddings of training, test, and out-of-distribution test data for reference (i.e., ResNet) and ALADIN (~5 hour)

```
cd benchmark
./create_embeddings.sh #change extension to .bat for Windows
cd ..
```

2. Create plots

```
python paper/plot-featurespace-ref.py
python paper/plot-featurespace-aladin.py
```

Resources:

- Main code ZIP file
- Pretrained weights of ALADIN (FigShare, 8.1Gb)
- Pretrained weights of Resnet and finetuned ECGFounder (FigShare, 580Mb)
- Jimenez-Perez model weights (FigShare, 251Mb)
- Stanford dataset (FigShare, 6Mb)
- Validation dataset (FigShare, 12Mb)
- MongoDB (FigShare, 5.3Gb)